

# EEE 475 - Medical Imaging Reconstruction and Processing

## Homework 5

Tuna Şahin, 22201730

### Contents

|           |                                                   |           |
|-----------|---------------------------------------------------|-----------|
| <b>0</b>  | <b>Introduction</b>                               | <b>1</b>  |
| 0.1       | GenAI Policy                                      | 1         |
| <b>1</b>  | <b>Part 1</b>                                     | <b>2</b>  |
| <b>2</b>  | <b>Part 2</b>                                     | <b>3</b>  |
| <b>3</b>  | <b>Part 3</b>                                     | <b>4</b>  |
| <b>4</b>  | <b>Part 4</b>                                     | <b>7</b>  |
| <b>5</b>  | <b>Part 5</b>                                     | <b>8</b>  |
| <b>6</b>  | <b>Part 6</b>                                     | <b>10</b> |
| <b>7</b>  | <b>Part 7</b>                                     | <b>11</b> |
| <b>8</b>  | <b>Part 8</b>                                     | <b>15</b> |
| <b>9</b>  | <b>Part 9</b>                                     | <b>17</b> |
| <b>10</b> | <b>Conclusion</b>                                 | <b>19</b> |
| <b>A</b>  | <b>Images for Each Iteration of Part 9</b>        | <b>20</b> |
| <b>B</b>  | <b>Question 8: Results for Alternate Sequence</b> | <b>23</b> |
| <b>C</b>  | <b>Question 9: Results for Alternate Sequence</b> | <b>24</b> |

### Introduction

This homework focuses on compressed sensing and reconstruction of randomly undersampled k-space data. Different methods are used such as Wavelet transforms and sparsity enforcing, SPIRiT, and  $l_1$  SPIRiT. The reconstruction for these methods were then compared using error maps and quantitative methods like PSNR and SSIM.

### GenAI Policy

ChatGPT 5.5 was used to assist in writing of the *spirit* function used. It was also used to debug certain implementations.

## Part 1

Firstly, the k-space data and the corresponding images were displayed for each coil. The fully sampled images were then reconstructed using sum-of-squares (SoS) reconstruction method and was assigned to be used as the reference image throughout the homework.

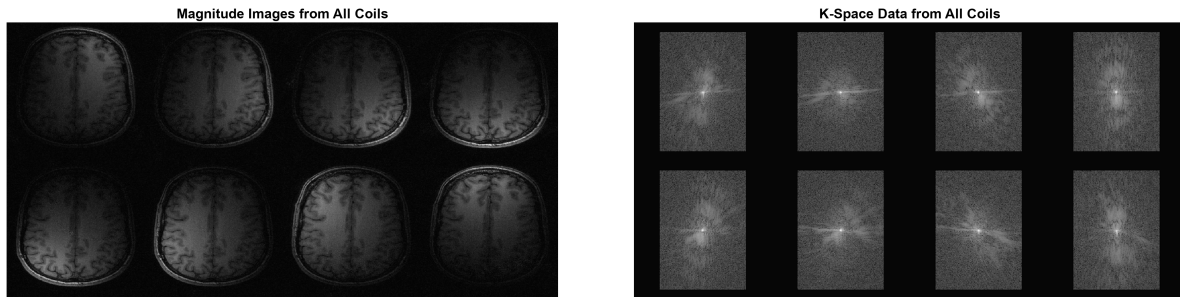


Figure 1.1: Provided Data in Spatial Domain and K-Space

Reference Image, Fully Sampled SoS Recon

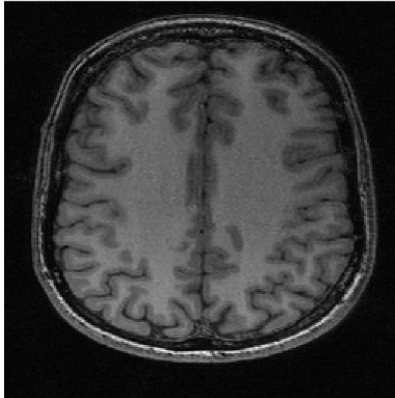


Figure 1.2: Reference Image

```
1 data = load("multicoil-random.mat");
2 im = data.im;
3 mask = data.mask;
4
5 figure()
6 figure('Units','pixels','Position',[100 100 1400 650]);
7 t = tiledlayout(1,2,'TileSpacing','compact','Padding','compact');
8 nexttile
9 montage(reshape(abs(im),[256, 256, 1, 8]),'displayRange',[],'size',[2 4]);
10 title("Magnitude Images from All Coils")
11 nexttile
12 montage(reshape(log10(abs(fft2c(im))+1),[256, 256, 1, 8]),'displayRange',[],'size',[2 4]);
13 title("K-Space Data from All Coils")
14 if save_image
15     exportgraphics(gcf,"q1_images.png",'Resolution',300)
16 end
17
18 figure()
19 m_sos = sqrt(sum(abs(im).^2,3));
20 m_ref = abs(m_sos)/prctile(abs(m_sos(:)),95); max_val = max(m_ref(:))
21 imshow(m_ref,[])
22 title("Reference Image, Fully Sampled SoS Recon")
23 if save_image
24     exportgraphics(gcf,"q1_mref.png",'Resolution',300)
25 end
```

## Part 2

The provided random undersampling mask is shown in Figure 2.1. The ratio of sampled pixels to the total number of pixels is:  $R = 4.9577$ . This mask is used to undersample all k-spaces for each coil and the resulting k-spaces and the corresponding images are shown in Figure 2.2. The resulting images appear much more noisy due to the undersampling artifacts. Compared to uniform undersampling the aliasing artifacts are incohesive where as the uniform undersampling artifacts appear as copies of the image. It is significantly easier to make out the original image by eye in this case.

The undersampled images are then combined using SoS to reconstruct the image in Figure 2.3. The corresponding error map is mostly noise and artifacts around sharp edges which is due to the undersampling mask being less dense at higher frequencies. The loss of high-frequency components result in loss of sharp edge information.

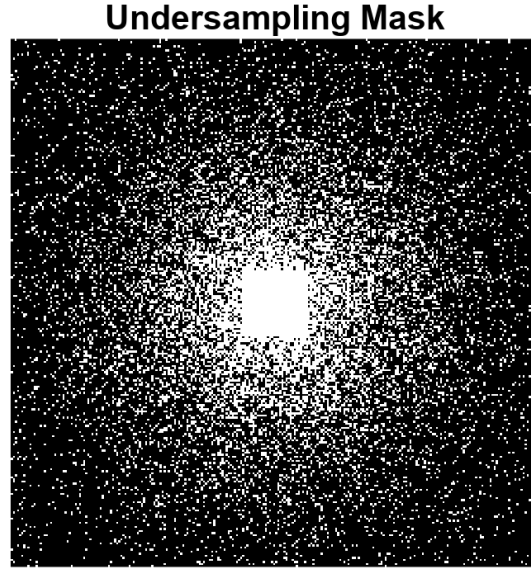


Figure 2.1: Undersampling Pattern Mask

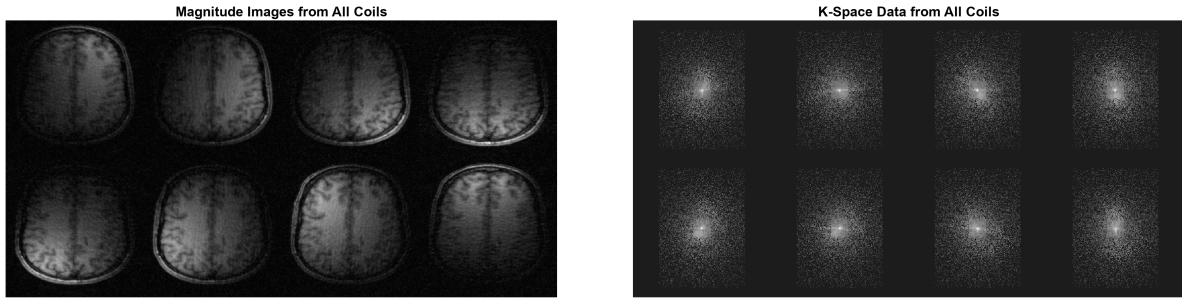


Figure 2.2: Undersampled Spatial Domain and K-Space Data

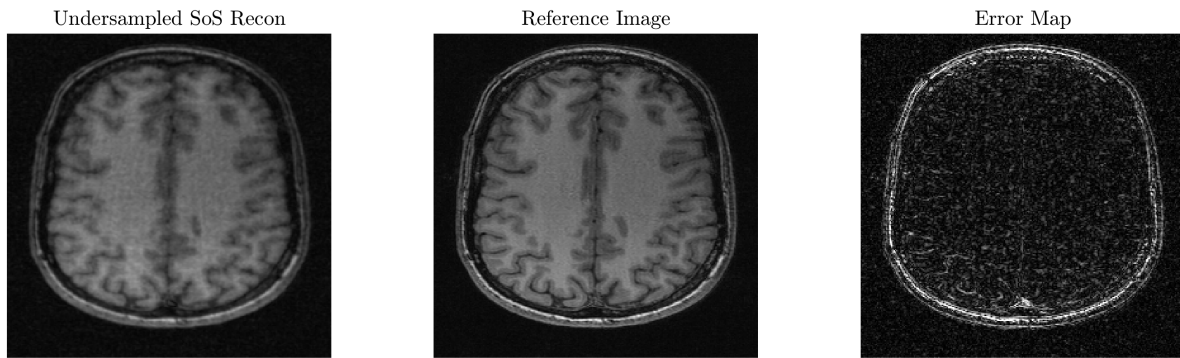


Figure 2.3: Comparison of Direct Reconstruction of Undersampled Image with the Reference Image

$$\text{PSNR} = 28.9548, \text{SSIM} = 0.8276$$

```

1 function [imu, Mu] = randundersample(im, mask)
2     Mu = fft2c(im).* mask;
3     imu = ifft2c(Mu);
4 end

```

```

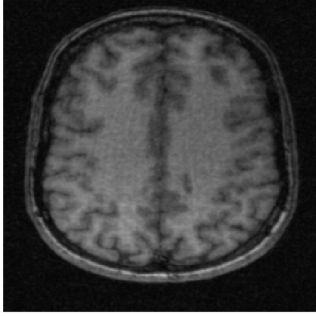
1 figure()
2 imshow(mask)
3 title("Undersampling Mask")
4 if save_image
5     exportgraphics(gcf,"q2_mask.png",'Resolution',300)
6 end
7
8 R = 1 / (sum(mask(:))/numel(mask))
9 [imu, Mu] = randundersample(im,mask);
10
11 figure()
12 figure('Units','pixels','Position',[100 100 1400 650]);
13 t = tiledlayout(1,2,'TileSpacing','compact','Padding','compact');
14 nexttile
15 montage(reshape(abs(imu),[256, 256, 1, 8]),'displayRange',[],'size',[2 4]);
16 title("Magnitude Images from All Coils")
17 nexttile
18 montage(reshape(log10(abs(Mu)+1),[256, 256, 1, 8]),'displayRange',[],'size',[2 4]);
19 title("K-Space Data from All Coils")
20 if save_image
21     exportgraphics(gcf,"q2_images.png",'Resolution',300)
22 end
23
24 figure()
25 mu_sos= sqrt(sum(abs(imu).^2,3));
26 mu_sos = abs(mu_sos)/prctile(abs(mu_sos(:)),95);
27 [PSNR2, SSIM2] = compare_images(mu_sos,m_ref,save_image,"q2_comp","Undersampled SoS
    ↳ Recon","Reference Image",max_val);

```

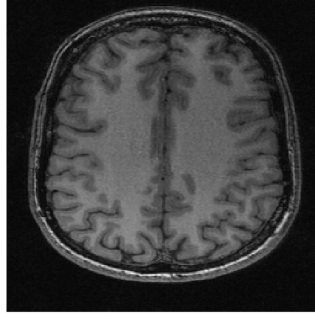
## Part 3

We will then enforce sparsity by filtering values below a certain intensity in the wavelet domain by using soft thresholding. We will then use SoS reconstruction to obtain images using these images.

Thresholded SoS Recon,  $\beta = 1.000 \times 10^{-10}$



Reference Image



Error Map

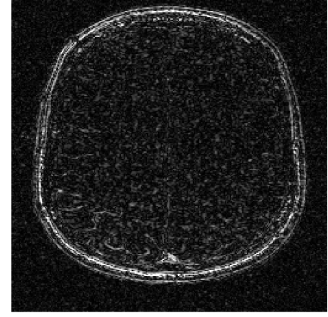
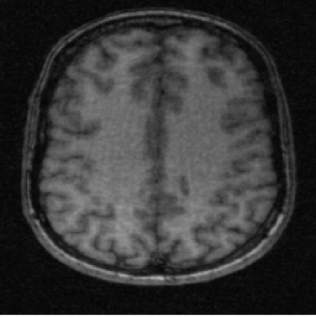
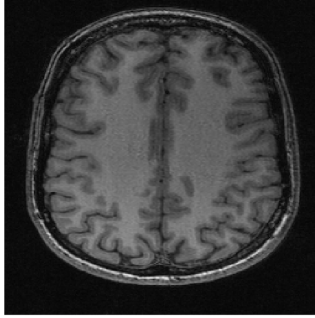


Figure 3.1: Reconstruction and Comparison of  $l_1$ -Regularized Image  $\beta = 10^{-10}$

Thresholded SoS Recon,  $\beta = 1.635 \times 10^{-6}$



Reference Image



Error Map

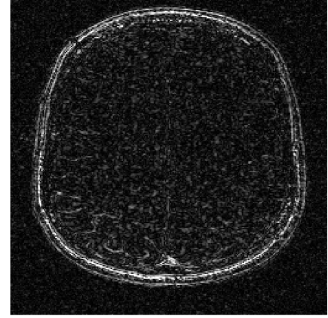
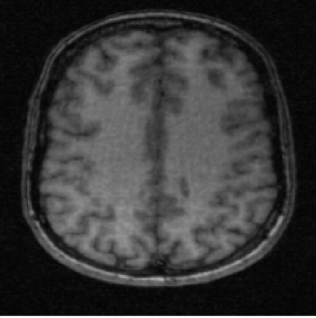
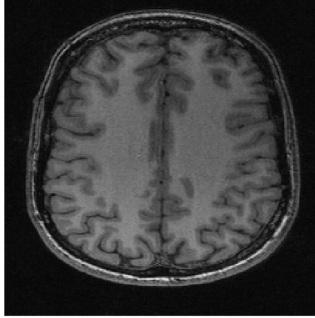


Figure 3.2: Reconstruction and Comparison of  $l_1$ -Regularized Image  $\beta = 1.635^{-6}$

Thresholded SoS Recon,  $\beta = 1.000 \times 10^{-3}$



Reference Image



Error Map

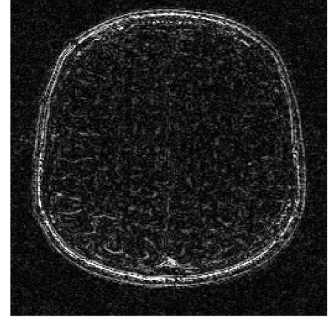
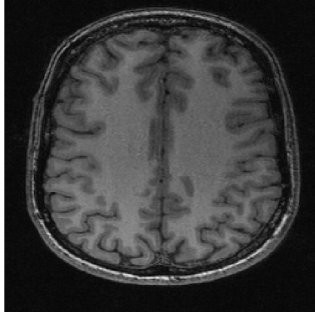


Figure 3.3: Reconstruction and Comparison of  $l_1$ -Regularized Image  $\beta = 10^{-3}$

Thresholded SoS Recon,  $\beta = 1.000 \times 10^{-2}$



Reference Image



Error Map

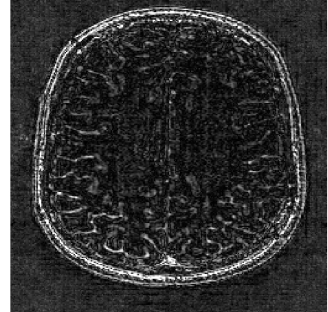


Figure 3.4: Reconstruction and Comparison of  $l_1$ -Regularized Image  $\beta = 10^{-2}$

| $\beta$                | PSNR    | SSIM   |
|------------------------|---------|--------|
| $10^{-10}$             | 28.9548 | 0.8276 |
| $1.635 \times 10^{-6}$ | 28.9545 | 0.8276 |
| $10^{-3}$              | 28.6854 | 0.8258 |
| $10^{-2}$              | 25.9804 | 0.6476 |

Table 3.1: Q3 PSNR and SSIM Values for Different  $\beta$  Values

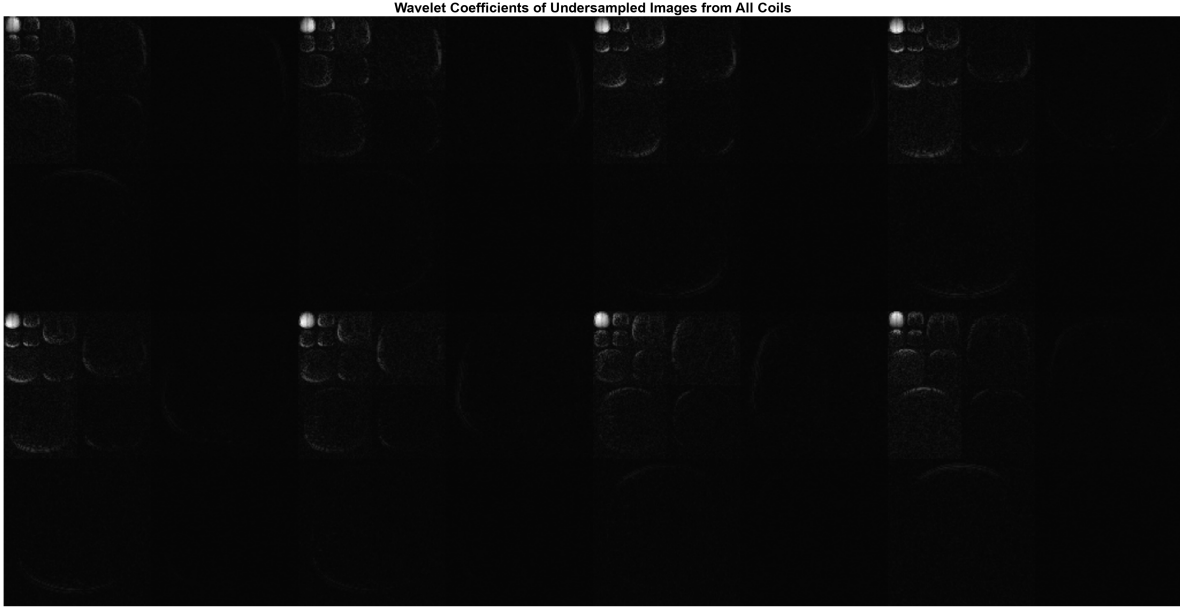


Figure 3.5: Wavelet Coefficients

```

1 function imth = llwavelet(imr,beta)
2     ww = Wavelet('Daubechies',4,4);
3     coeffW = ww*imr;
4     mag = abs(coeffW);
5     coeffW = coeffW./(mag+eps) .* max(0,mag - beta*max(mag(:)));
6     imth = ww' * coeffW;
7 endss

```

```

1  ww_coefs = zeros(size(im));
2  ww = Wavelet('Daubechies',4,4);
3  for i = 1:8
4      image = imu(:, :, i);
5      ww_coefs(:, :, i) = log10(abs(ww*image)+1);
6  end
7
8  figure()
9  figure('Units','pixels','Position',[0 0 1400 750]);
10 montage(reshape(ww_coefs,[256, 256, 1, 8]),'displayRange',[],'size',[2 4]);
11 title("Wavelet Coefficients of Undersampled Images from All Coils")
12 if save_image
13     exportgraphics(gcf,"q3_wavelet.png",'Resolution',300)
14 end
15 betas = [10^-10 1.635*10^-6 10^-3 10^-2];
16 PSNRs3 = zeros(size(betas));
17 SSIMs3 = zeros(size(betas));
18 imths = num2cell(zeros(size(betas)));
19 for j = 1:numel(betas)
20     imth = zeros(size(im));
21     for i = 1:8
22         imth(:, :, i) = l1wavelet(imu(:, :, i), betas(j));
23     end
24     mth_sos = sqrt(sum(abs(imth).^2,3));
25     mth_sos = abs(mth_sos)/prctile(abs(mth_sos(:)),95);
26     imths{j} = mth_sos;
27     [PSNRs3(j), SSIMs3(j)] = compare_images(mth_sos, m_ref, save_image, ...
28         strcat("q3_comp_", num2str(j)), ...
29         strcat("Thresholded SoS Recon, $\beta$ = ", num2sci(betas(j), "latex")), ...
30         "Reference Image", max_val);
31 end

```

## Part 4

Similar to the GRAPPA kernel function in HW4, we approximated the SPIRiT kernel using a  $32 \times 32$  calibration region at the center of the k-space. The kernel is shown in Figure 4.1.

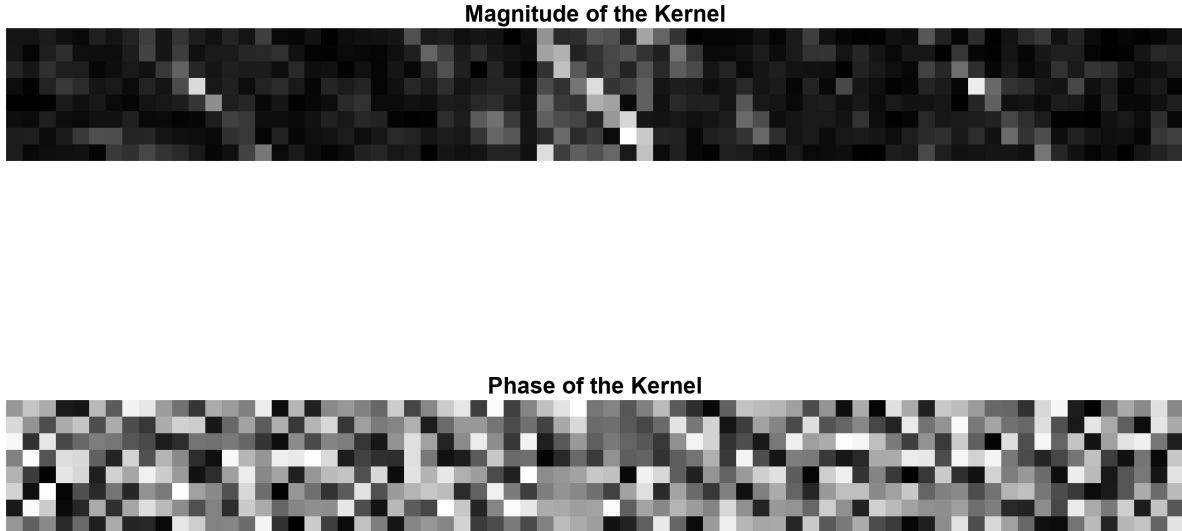


Figure 4.1: SPIRiT Kernel

```

1 function Mc = imcalib(im, calibx, caliby)
2     [Nx, Ny, Nc] = size(im);
3     k_data = fft2c(im);
4     cx = floor(Nx/2) + 1;
5     cy = floor(Ny/2) + 1;
6     x_range = cx - floor(calibx/2) : cx + floor(calibx/2) - 1;
7     y_range = cy - floor(caliby/2) : cy + floor(caliby/2) - 1;
8     Mc = k_data(x_range, y_range, :);
9 end

```

```

1 function kernel = calibrateSpirit(Mc, lambda)
2     [Nx, Ny, Nc] = size(Mc);
3     calib_mask = any(abs(Mc) > 0, 3);
4     [xs, ys] = find(calib_mask);
5     if isempty(xs)
6         error('Calibration data Mc appears to be empty.');
```

```

7     end
8     xmin = min(xs); xmax = max(xs);
9     ymin = min(ys); ymax = max(ys);
10    P = 9*Nc - 1;
11    kernel = zeros(P, Nc);
12    for target_coil = 1:Nc
13        S = []; T = [];
14        for x = xmin+1:xmax-1
15            for y = ymin+1:ymax-1
16                src = [];
17                for dx = -1:1
18                    for dy = -1:1
19                        xx = x + dx; yy = y + dy;
20                        for coil = 1:Nc
21                            if dx == 0 && dy == 0 && coil == target_coil
22                                continue;
23                            end
24                            src = [src; Mc(xx, yy, coil)];
25                        end
26                    end
27                end
28                tgt = Mc(x, y, target_coil);
29                S = [S; src.']; T = [T; tgt];
30            end
31        end
32        kernel(:, target_coil) = (S' * S + lambda * eye(P)) \ (S' * T);
33    end
34 end

```

```

1 Mc = imcalib(im, 32, 32); %from hu4
2 calibrateSpirit(Mc, 0);
3 kernel = calibrateSpirit(Mc, 0);
4 figure()
5 figure('Units','pixels','Position',[100 100 1400 650]);
6 t = tiledlayout(2,1,'TileSpacing','compact','Padding','compact');
7 nexttile
8 imshow(abs(kernel.'),[])
9 title("Magnitude of the Kernel")
10 nexttile
11 imshow(angle(kernel.'),[])
12 title("Phase of the Kernel")
13 if save_image
14     exportgraphics(gcf,"q4_kernel.png",'Resolution',300)
15 end

```

## Part 5

Using the kernel obtained in part 4, we will then fill the empty samples in the k-space. We will also use a regularised version again by selecting a regularization parameter  $\lambda$ . A single pass of the SPIRiT

method was performed for this question without enforcing data consistency and the SoS recon results are shown in Figure 5.1.

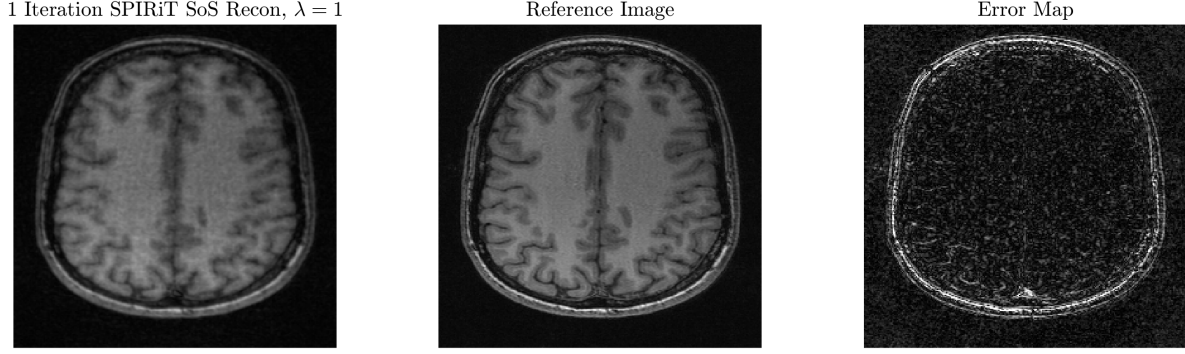


Figure 5.1: Comparison of a Single SPIRiT Pass Reconstruction

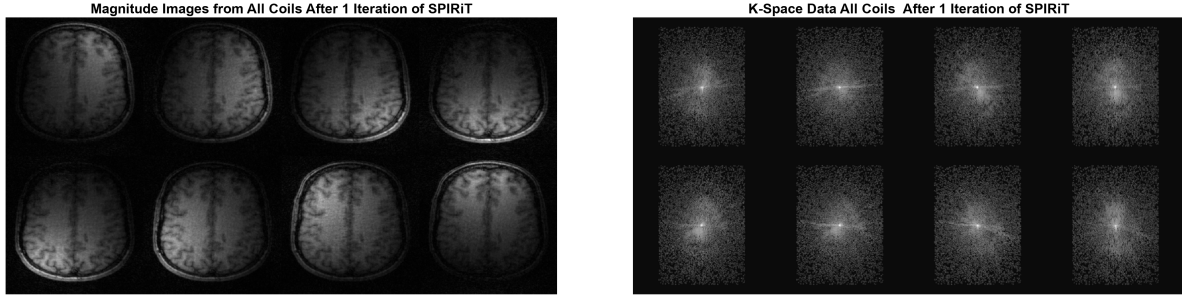


Figure 5.2: Spatial-Domain and K-Space Data of Single SPIRiT Pass Data

PSNR = 28.5061, SSIM = 0.8101

```

1 function [imr, Mr] = spirit(Mu, kernel)
2     [Nx, Ny, Nc] = size(Mu);
3     P = 9*Nc - 1;
4     if size(kernel,1) ~= P || size(kernel,2) ~= Nc
5         error('kernel must have size (9*Nc - 1) x Nc.');
```

```

1 lambda = 1;
2 kernel_regularised = calibrateSpirit(Mc, lambda);
3 [imr, Mr] = spirit(Mu, kernel_regularised);
4 mspi_sos = sqrt(sum(abs(imr).^2,3));
5 mspi_sos = abs(mspi_sos)/prctile(abs(mspi_sos(:)),95);
6 [PSNR5, SSIM5] = compare_images(mspi_sos,m_ref,save_image,"q5_comp","1 Iteration SPIRiT SoS
  ↳ Recon, $\lambda = 1$", "Reference Image",max_val);
7 figure()
8 figure('Units','pixels','Position',[100 100 1400 650]);
9 t = tiledlayout(1,2,'TileSpacing','compact','Padding','compact');
10 nexttile
11 montage(reshape(abs(imr),[256, 256, 1, 8]),'displayRange',[],'size',[2 4]);
12 title("Magnitude Images from All Coils After 1 Iteration of SPIRiT")
13 nexttile
14 montage(reshape(log10(abs(Mr)+1),[256, 256, 1, 8]),'displayRange',[],'size',[2 4]);
15 title("K-Space Data All Coils After 1 Iteration of SPIRiT")
16 if save_image
17     exportgraphics(gcf,"q5_images.png",'Resolution',300)
18 end

```

## Part 6

Then data consistency was enforced after a single pass of the SPIRiT algorithm. This condition significantly improved SSIM and PSNR values since we projected the image onto a set that contains the original data. Again the images and the SoS reconstruction is provided.

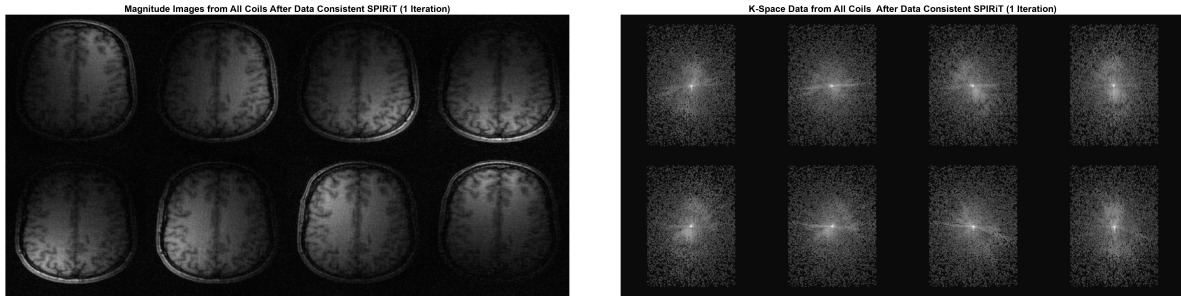


Figure 6.1: Spatial-Domain and K-Space Data for Data Consistent Single SPIRiT Data

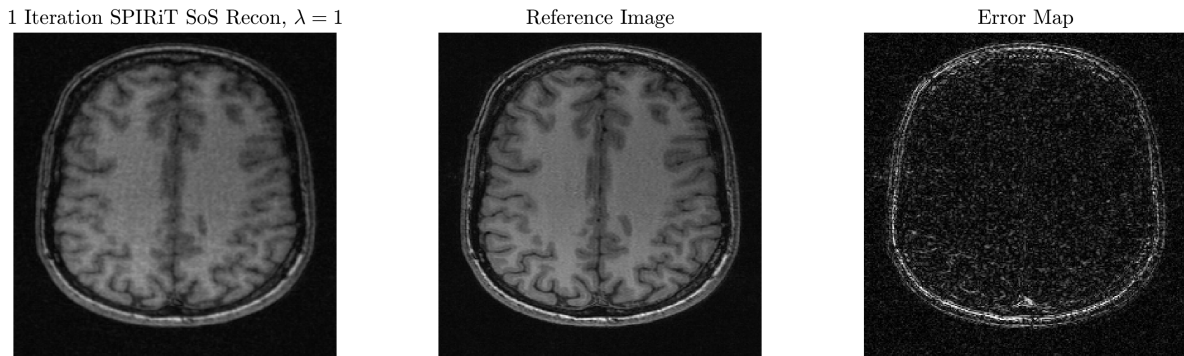


Figure 6.2: Comparison for Data Consistent Single SPIRiT Data

$$\text{PSNR} = 30.4275, \text{SSIM} = 0.8538$$

```

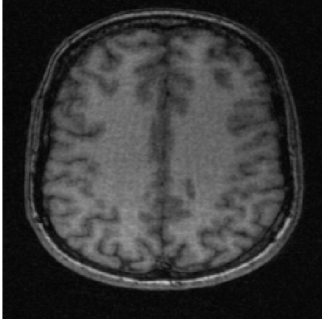
1 lambda = 1;
2 kernel_regularised = calibrateSpirit(Mc, lambda);
3 [imr, Mr_con] = spirit(Mu, kernel_regularised);
4 Mr_con(repmat(mask, [1 1 size(Mr_con,3)])) = Mu(repmat(mask, [1 1 size(Mr_con,3)]));
5 imr_con = ifft2c(Mr_con);
6 mcon_sos = sqrt(sum(abs(imr_con).^2,3));
7 mcon_sos = abs(mcon_sos)/prctile(abs(mcon_sos(:)),95);
8 [PSNR6, SSIM6] = compare_images(mcon_sos,m_ref,save_image,"q5_comp","1 Iteration SPIRiT SoS
   ↳ Recon,  $\lambda = 1$ ","Reference Image",max_val);
9 figure()
10 figure('Units','pixels','Position',[100 100 2000 650]);
11 t = tiledlayout(1,2,'TileSpacing','compact','Padding','compact');
12 nexttile
13 montage(reshape(abs(imr_con),[256, 256, 1, 8]),'displayRange',[],'size',[2 4]);
14 title("Magnitude Images from All Coils After Data Consistent SPIRiT (1 Iteration)")
15 nexttile
16 montage(reshape(log10(abs(Mr_con)+1),[256, 256, 1, 8]),'displayRange',[],'size',[2 4]);
17 title("K-Space Data from All Coils After Data Consistent SPIRiT (1 Iteration)")
18 if save_image
19     exportgraphics(gcf,"q6_images.png",'Resolution',300)
20 end

```

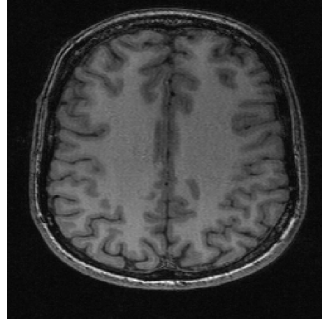
## Part 7

A single iteration performed in part 6 was then scaled to perform multiple iterations (10 to be exact). Since this is a POCS-type algorithm, the iterative projection onto different sets converges the result to the true image with each iteration. The resulting SoS images, PSNR, and SSIM values are provided for each iteration, allowing for observations regarding the changes through each iteration. This iterative process increased the SSIM and PSNR values even further upon the increase in part 6.

1 Iterations SPIRiT SoS Recon,  $\lambda = 1$



Reference Image



Error Map

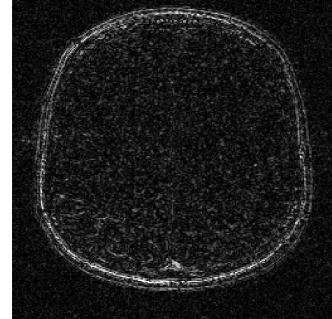
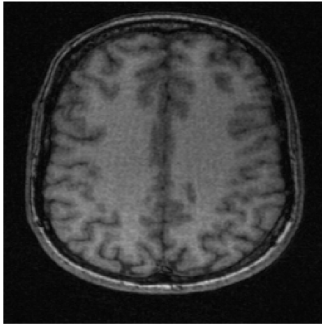
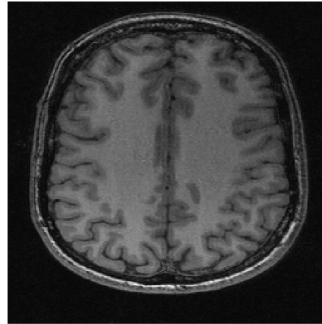


Figure 7.1: Data Consistent SPIRiT, Iteration: 1/10

2 Iterations SPIRiT SoS Recon,  $\lambda = 1$



Reference Image



Error Map

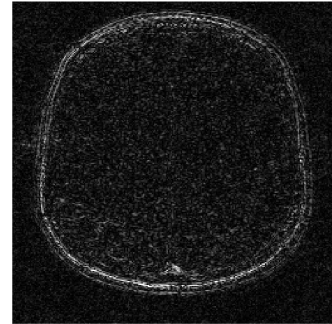
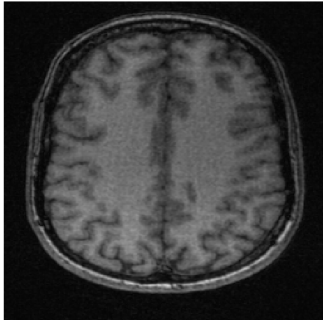
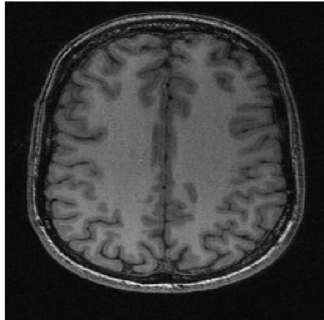


Figure 7.2: Data Consistent SPIRiT, Iteration: 2/10

3 Iterations SPIRiT SoS Recon,  $\lambda = 1$



Reference Image



Error Map

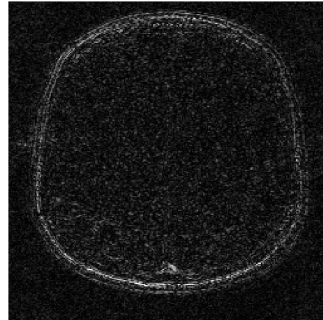
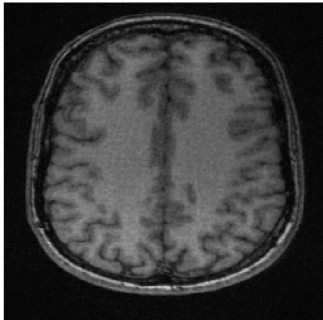
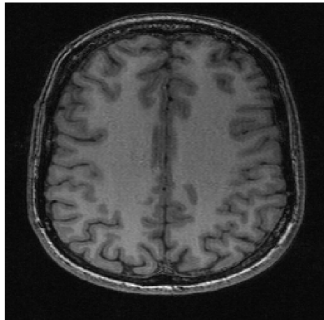


Figure 7.3: Data Consistent SPIRiT, Iteration: 3/10

4 Iterations SPIRiT SoS Recon,  $\lambda = 1$



Reference Image



Error Map

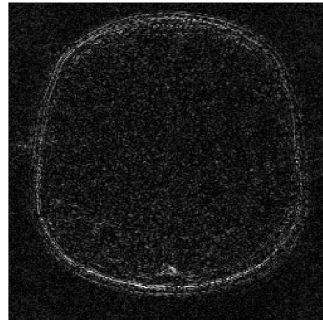
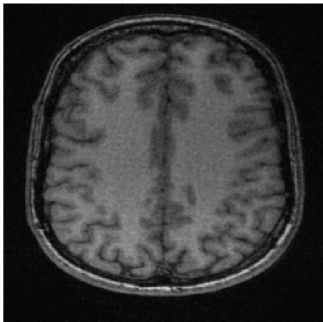
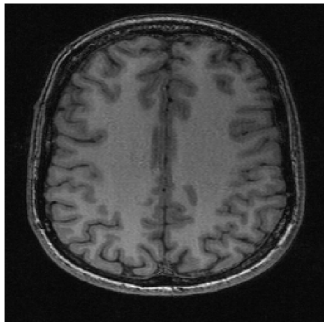


Figure 7.4: Data Consistent SPIRiT, Iteration: 4/10

5 Iterations SPIRiT SoS Recon,  $\lambda = 1$



Reference Image



Error Map

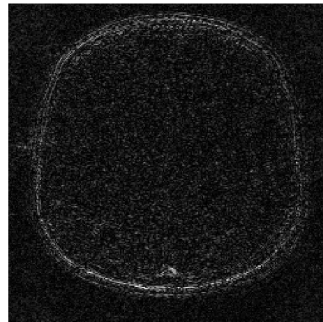
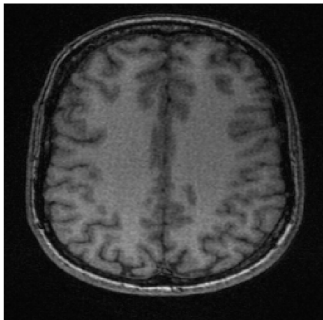
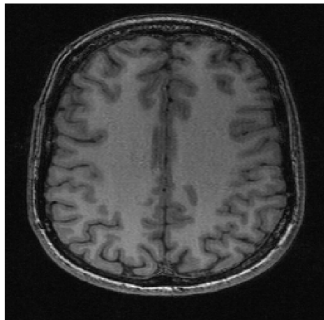


Figure 7.5: Data Consistent SPIRiT, Iteration: 5/10

6 Iterations SPIRiT SoS Recon,  $\lambda = 1$



Reference Image



Error Map

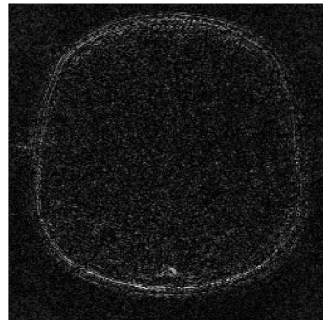
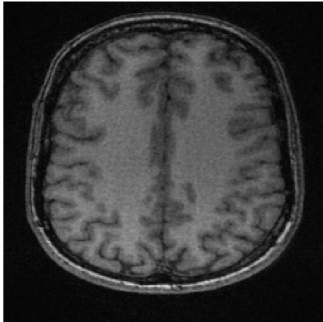
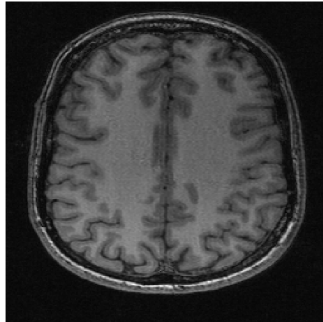


Figure 7.6: Data Consistent SPIRiT, Iteration:6/10

7 Iterations SPIRiT SoS Recon,  $\lambda = 1$



Reference Image



Error Map

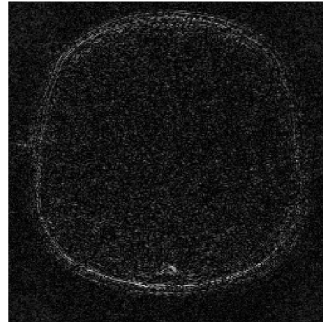
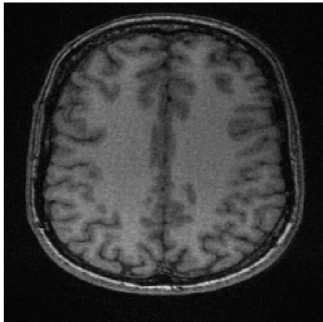
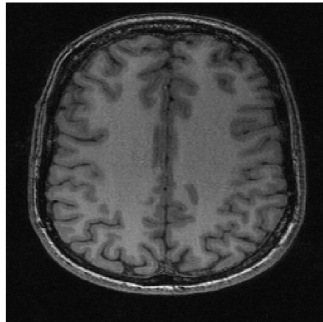


Figure 7.7: Data Consistent SPIRiT, Iteration: 7/10

8 Iterations SPIRiT SoS Recon,  $\lambda = 1$



Reference Image



Error Map

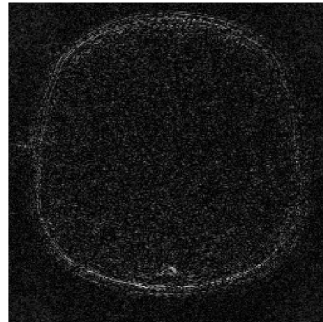
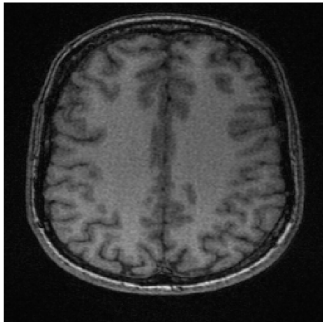
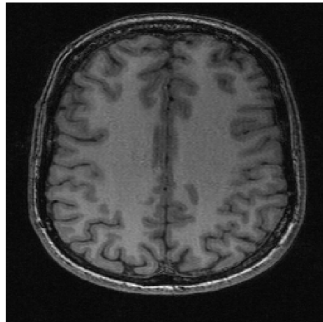


Figure 7.8: Data Consistent SPIRiT, Iteration: 8/10

9 Iterations SPIRiT SoS Recon,  $\lambda = 1$



Reference Image



Error Map

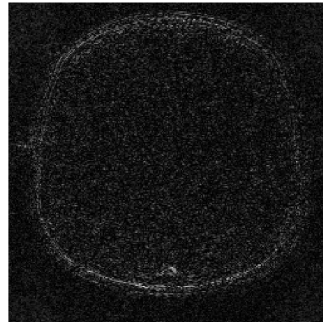
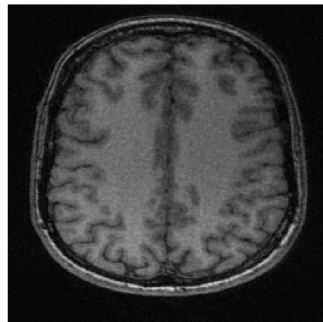
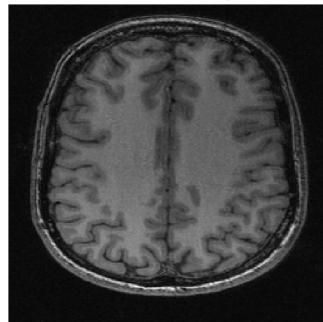


Figure 7.9: Data Consistent SPIRiT, Iteration: 9/10

10 Iterations SPIRiT SoS Recon,  $\lambda = 1$



Reference Image



Error Map

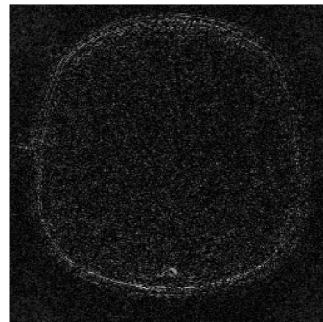


Figure 7.10: Data Consistent SPIRiT, Iteration: 10/10

| Metric | 1       | 2       | 3       | 4       | 5       | 6       | 7       | 8       | 9       | 10      |
|--------|---------|---------|---------|---------|---------|---------|---------|---------|---------|---------|
| PSNR   | 30.4275 | 31.1207 | 31.6846 | 32.0274 | 32.3289 | 32.5114 | 32.6806 | 32.7698 | 32.8694 | 32.9040 |
| SSIM   | 0.8538  | 0.8629  | 0.8682  | 0.8703  | 0.8720  | 0.8722  | 0.8726  | 0.8720  | 0.8718  | 0.8708  |

Table 7.1: Q7 PSNR and SSIM Values for Each Iteration

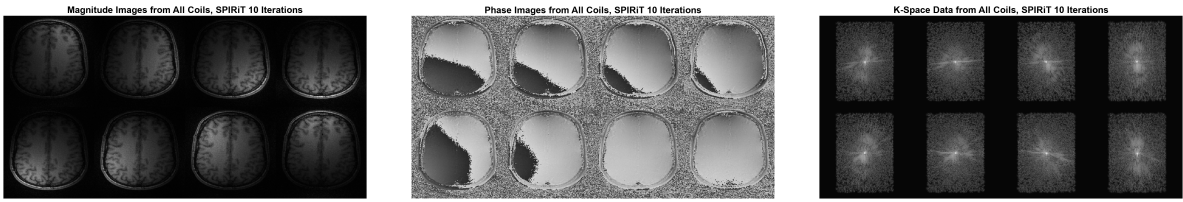


Figure 7.11: Spatial-Domain and K-Space Data for 10 Iterations of SPIRiT

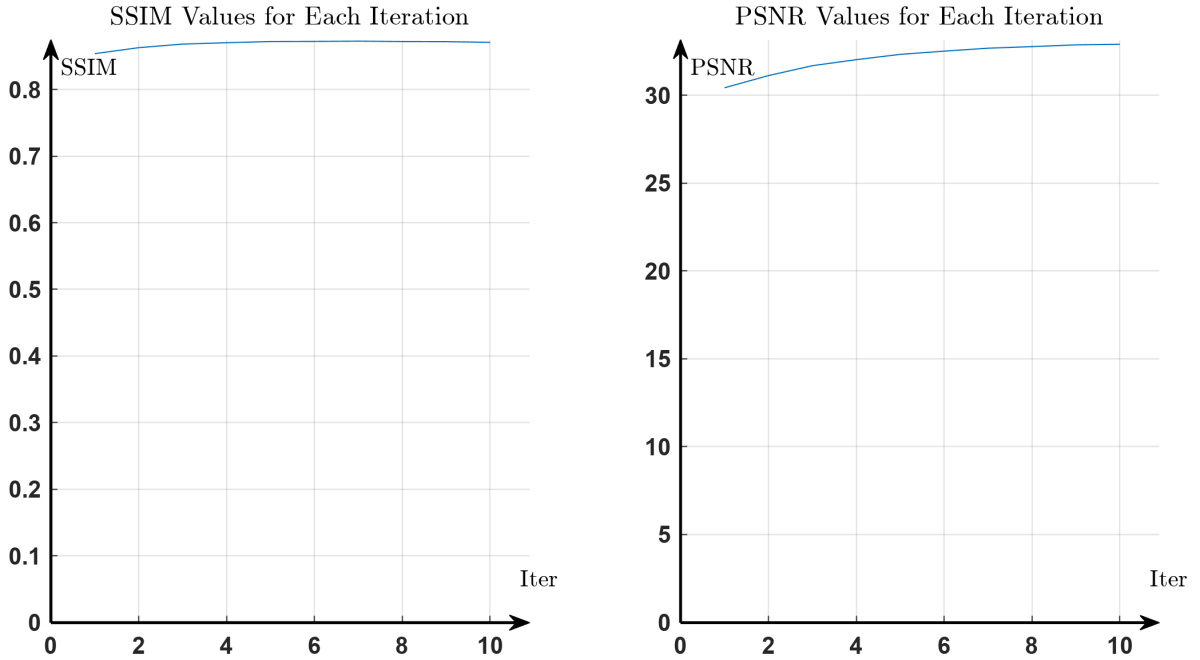


Figure 7.12: SSIM and PSNR Values for Each SPIRiT Iteration

```

1 PSNRs7 = zeros([1 10]);
2 SSIMs7 = zeros([1 10]);
3 lambda = 1;
4 kernel_regularised = calibrateSpirit(Mc, lambda);
5 [imu, Mu] = randundersample(im,mask);
6 M7 = Mu;
7 Niter = 10;
8 imr7 = zeros(size(Mu,1), size(Mu,2), size(Mu,3), Niter);
9 Mr7 = zeros(size(Mu,1), size(Mu,2), size(Mu,3), Niter);
10 mask2 = repmat(mask, [1 1 size(Mu,3)]);
11
12 M_loop = Mu;
13 for i = 1:Niter
14     [imr_loop, M_loop] = spirit(M_loop, kernel_regularised);
15     M_loop(mask2) = Mu(mask2);
16     imr_con = ifft2c(M_loop);
17     mcon_sos = sqrt(sum(abs(imr_con).^2,3));
18     mcon_sos = abs(mcon_sos)/prctile(abs(mcon_sos(:)),95);
19     [PSNRs7(i), SSIMs7(i)] = compare_images(mcon_sos,m_ref,save_image, strcat("q7_comp_",num2str(
        ↪ r(i)),strcat(num2str(i)," Iterations SPIRiT SoS Recon, $\lambda = 1$"),"Reference
        ↪ Image",max_val);
20 end
21 figure()
22 figure('Units','pixels','Position',[100 100 2000 650]);
23 t = tiledlayout(1,3,'TileSpacing','compact','Padding','compact');
24 nexttile
25 montage(reshape(abs(imr_loop),[256, 256, 1, 8]),'displayRange',[],'size',[2 4]);
26 title("Magnitude Images from All Coils, SPIRiT 10 Iterations")
27 nexttile
28 montage(reshape(angle(imr_loop),[256, 256, 1, 8]),'displayRange',[],'size',[2 4]);
29 title("Phase Images from All Coils, SPIRiT 10 Iterations")
30 nexttile
31 montage(reshape(log10(abs(M_loop)+1),[256, 256, 1, 8]),'displayRange',[],'size',[2 4]);
32 title("K-Space Data from All Coils, SPIRiT 10 Iterations")
33 if save_image
34     exportgraphics(gcf,"q7_images.png",'Resolution',300)
35 end
36 figure()
37 subplot(1,2,1)
38 fineplot(1:10,SSIMs7,"SSIM Values for Each Iteration","Iter","SSIM",[],[],"off",[800
    ↪ 400],"-","-");
39 subplot(1,2,2)
40 fineplot(1:10,PSNRs7,"PSNR Values for Each Iteration","Iter","PSNR",[],[],"off",[800
    ↪ 400],"-","-");
41 if save_image
42     exportgraphics(gcf,"q7_ssimsnr.png",'Resolution',300)
43 end

```

## Part 8

To enforce sparsity in spatial domain, wavelet domain thresholding was implemented into the existing SPIRiT algorithm. This improved the PSNR and SSIM values compared to the single iteration without sparsity in part 6. For this question and question 9, the k-space data consistency was told to be implemented before the application of the SPIRiT kernel. However, applying the data consistency step after the kernel increases performance significantly. Consequently, both the instructed version and the alternate versions results are available. The results for the alternate sequence are available in Appendix B.